VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.



NATURAL LANGUAGE PROCESSING
AAT REPORT
on

NLP-Based Customer Support Ticket Classification System Using Machine Learning

Submitted by

DIPTANSHU SHEKHAR (1BM23CS361)
VAIBHAV DHAR (1BM23CS358)
MADHAN K (1BM23CS173)
PRAVEEN RATHOD (1BM23CS248)

Under the Guidance of
Dr. Prameetha Pai
Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Aug 2026-Nov 2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the AAT work entitled "**NLP-Based Customer Support Ticket Classification System Using Machine Learning**" is carried out by **DIPTANSHU SHEKHAR (1BM23CS361), VAIBHAV DHAR (1BM23CS358), MADHAN K (1BM23CS173) and PRAVEEN RATHOD (1BM23CS248)** who are bonafide students of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visveswaraya Technological University, Belgaum during the year 2026-2027. The AAT report has been approved as it satisfies the academic requirements in respect of **Natural Language Processing (23CS7PENLP)** work prescribed for the said degree.

Signature of the Guide
Dr. Prameetha Pai
Assistant Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Latha N R
Prof.& Head Cluster D, Dept. of CSE
BMSCE, Bengaluru

B.M.S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, DIPTANSHU SHEKHAR (1BM23CS361), VAIBHAV DHAR (1BM23CS358), MADHAN K (1BM23CS173) and PRAVEEN RATHOD (1BM23CS248), students of 7th Semester, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this AAT entitled "NLP-Based Customer Support Ticket Classification System Using Machine Learning" has been carried out by us under the guidance of Dr. Prameetha Pai, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester August 2026 to November 2026.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

DIPTANSHU SHEKHAR (1BM23CS361)

VAIBHAV DHAR (1BM23CS358)

MADHAN K (1BM23CS173)

PRAVEEN RATHOD (1BM23CS248)

TABLE OF CONTENTS

Chapter no.	Title	Page no.
1	**PROBLEM STATEMENT**	**1**
1.1	Motivation and Applications	1
1.2	Work Highlights	2
2	**DESIGN APPROACH**	**3**
2.1	NLP Pipeline Design	3
2.1.1	Dataset Description	4
2.1.2	Text Preprocessing using NLTK	5
2.1.3	Feature Extraction using TF-IDF	6
2.1.4	Classification using Linear SVM	8
2.2	Validation and Evaluation Design	9
2.2.1	Data Splitting and Cross-Validation	9
2.2.2	Evaluation Metrics	10
3	**System Requirements**	**11**
3.1	Hardware Requirements	11
3.2	Software Requirements (Libraries used)	11
4	**IMPLEMENTATION**	**13**
4.1	Pseudocode / Algorithm	14

4.2	Output (Screenshots)	15
5	**CONCLUSION**	**29**
	REFERENCES	**30**

1. PROBLEM STATEMENT

Customer support teams in e-commerce, banking, telecommunication and similar service companies receive thousands of free-text requests every day through e-mail, live chat and web forms. Each request has to be read and forwarded (routed) to the team that can solve it, for example billing, deliveries, refunds or account management. Doing this manually is slow and expensive, and a wrongly routed ticket waits in the wrong queue, which increases the time taken to resolve the customer's problem.

Problem statement: Given the text of a customer's support message, automatically predict which of 11 support categories it belongs to (ACCOUNT, CANCEL, CONTACT, DELIVERY, FEEDBACK, INVOICE, ORDER, PAYMENT, REFUND, SHIPPING and SUBSCRIPTION), report how confident the prediction is, and flag uncertain messages for manual review by a human agent.

This is a supervised multi-class text classification problem, which is a core application of Natural Language Processing (NLP) [9]. The approach chosen in this work is a classical NLP and machine learning pipeline: every message is cleaned and normalised with the **NLTK** library [2] (tokenization, stopword removal and WordNet lemmatization [8]), converted into numerical feature vectors using **TF-IDF** weighting [6], [10], and classified by a **Linear Support Vector Machine (SVM)** [4], [5]. The model is trained and evaluated on the publicly available Bitext Customer Support dataset [1] of 26,872 labelled customer messages, and it is deployed in an interactive web application built with Streamlit [13].

1.1 Motivation and Applications

Customer messages are unstructured natural language. The same request can be written in many ways ("I want my money back", "get a refund", "reimbursement"), messages contain spelling mistakes and informal language, and small words such as "not" can change the meaning completely ("my parcel has not arrived"). Hand-written keyword rules break easily on such text, while a model that learns from labelled examples can generalise to new phrasings. This makes ticket routing a natural and practical problem for NLP.

The combination of TF-IDF features and a linear SVM was chosen because it is one of the strongest classical methods for short-text classification [5]: it handles thousands of sparse word

features well, trains in a short time on a normal laptop CPU without a GPU, and is easy to interpret, because every word receives a weight for every category. Typical applications of such a system are:

- **Automatic ticket routing** in help-desk software, so that each request reaches the correct team immediately.
- **Front-line triage for chatbots and virtual assistants**, deciding which flow or department should handle a user's request.
- **E-mail sorting** for banks, telecom operators and online stores that receive large volumes of customer mail.
- **Support analytics**, i.e. measuring which kinds of problems customers report most often so that staffing and products can be improved.
- **Workload reduction**, where confident predictions are routed automatically and only uncertain messages are reviewed by a human.

1.2 Work Highlights

- Inspected and used the **original Bitext Customer Support dataset** [1] (26,872 messages, 11 categories). The actual schema was verified from the downloaded file, and the agent-response and intent columns were excluded from the input to avoid data leakage.
- Removed **2,598 repeated messages** before splitting the data, so that the same sentence can never appear in both the training set and the test set.
- Implemented a reusable **NLTK preprocessing pipeline** (tokenization, stopword removal that keeps negation words, WordNet lemmatization). It is stored inside the saved model, so training and live prediction always process text in exactly the same way.
- Built a **TF-IDF + Linear SVM** classification pipeline with calibrated confidence scores.
- Validated the model with **5-fold cross-validation** (mean macro-F1 = 0.9985 ± 0.0004) and a separate validation set, and evaluated it once on a held-out test set of 3,642 unseen messages: **accuracy = 99.78%, macro-F1 = 0.9978**, with only 8 misclassified messages.
- Added **manual-review flagging** for low-confidence or unknown messages, and developed a **Streamlit web application** with five pages: Home, Dataset Explorer, Classify a Ticket, Model Performance and Session Analytics.

2. DESIGN APPROACH

The system is designed as a modular NLP pipeline. Each stage is implemented as a separate Python module, and the stages that transform text (NLTK preprocessing, TF-IDF and the Linear SVM) are combined into a single Scikit-learn Pipeline object [3]. Figure 2.1 shows the overall architecture. The labelled dataset is loaded and cleaned, split into training, validation and test sets, and the training messages are used to fit the pipeline. The fitted pipeline is validated, evaluated and saved to disk; the prediction module and the web application then load the saved pipeline to classify new messages without retraining.

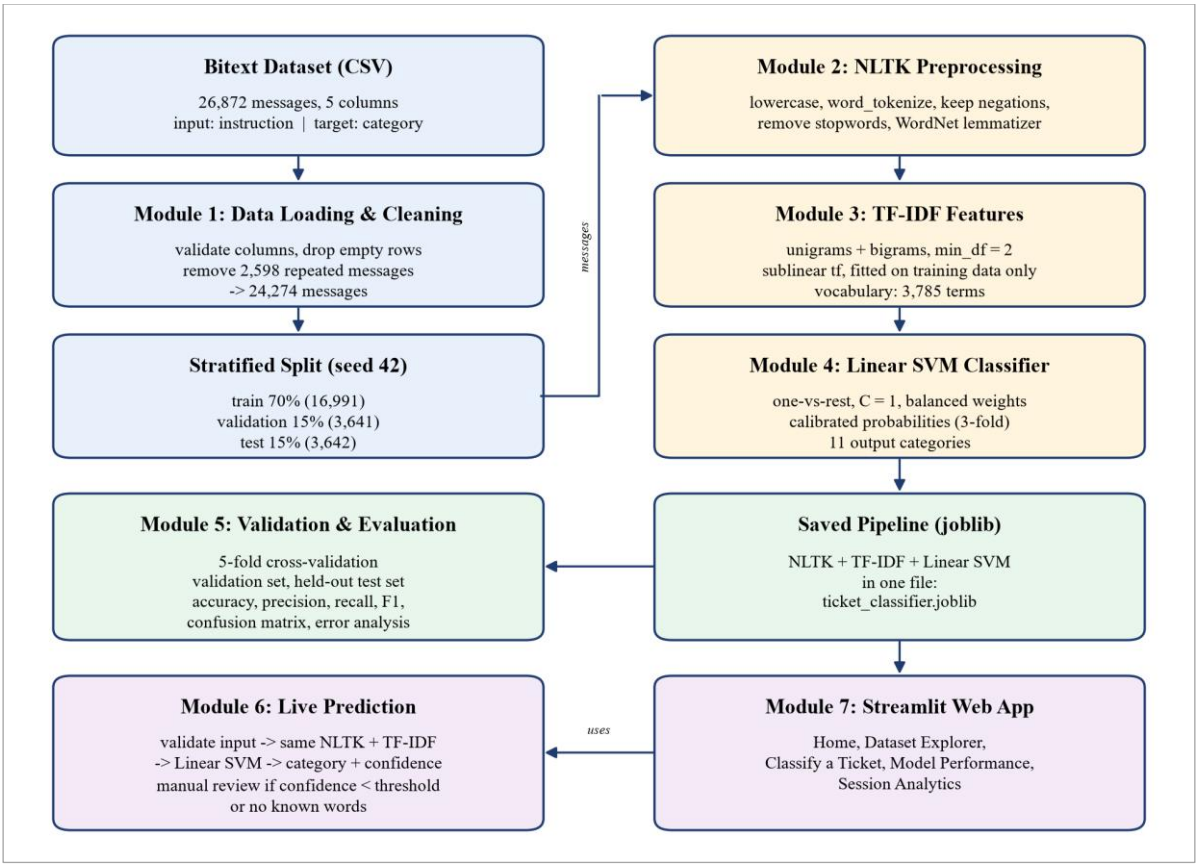


Figure 2.1: System architecture of the ticket classification system

2.1 NLP Pipeline Design

The design follows the standard text classification workflow: data → preprocessing → feature extraction → classification. Because the NLTK preprocessing function is plugged into the TF-IDF vectorizer, and the vectorizer and classifier are steps of one pipeline, all three are fitted together on the training data only and saved together in one file (ticket_classifier.joblib). This guarantees

that a new message is processed in exactly the same way as the training data, and it prevents information from the validation and test sets from leaking into training.

2.1.1 Dataset Description

The project uses the **Bitext Customer Service Tagged Training Dataset for LLM-based Virtual Assistants** [1], published on GitHub under the CDLA-Sharing-1.0 license. The file Bitext_Sample_Customer_Support_Training_Dataset_27K_responses-v11.csv (19.2 MB) contains 26,872 rows and 5 columns, with no missing values. The fields and their use in this project are listed in Table 2.1.

Table 2.1: Fields of the Bitext dataset and their use

Column	Meaning	Use in this project
flags	Language-variation tags (e.g. typos, colloquial language)	Not used
instruction	The customer's message	Model input
category	High-level support category (11 classes)	Model target
intent	Fine-grained intent (27 classes) within a category	Not used (it directly reveals the category)
response	Example reply written by a support agent	Not used (written after knowing the category)

The 11 target categories and the number of messages in each are shown in Table 2.2. The category names were taken from the CSV file itself: the file uses CANCEL and SHIPPING, whereas the repository README calls them CANCELLATION_FEE and SHIPPING_ADDRESS.

Table 2.2: Target categories (before cleaning)

Category	Requests covered (intents)	Messages	Share
ACCOUNT	create, delete, edit, switch account; recover password; registration problems	5,986	22.3%
ORDER	place, cancel, change, track order	3,988	14.8%
REFUND	check refund policy, get refund, track refund	2,992	11.1%
INVOICE	check invoice, get invoice	1,999	7.4%
CONTACT	contact customer service, contact human agent	1,999	7.4%
PAYMENT	check payment methods, payment issue	1,998	7.4%
FEEDBACK	complaint, review	1,997	7.4%
DELIVERY	delivery options, delivery period	1,994	7.4%

Category	Requests covered (intents)	Messages	Share
SHIPPING	change or set up shipping address	1,970	7.3%
SUBSCRIPTION	newsletter subscription	999	3.7%
CANCEL	check cancellation fee	950	3.5%
Total		26,872	100%

Data quality findings. The dataset has no missing values. 2,598 messages occur more than once (with different example responses); all copies carry the same label, and they were removed before splitting, leaving **24,274 unique messages**. 6,670 messages contain entity placeholders such as {{Order Number}}, and many contain deliberate spelling mistakes (e.g. "hw can I modify purchase") that Bitext added to imitate real users. The average message is short (8.7 words). The classes are imbalanced: ACCOUNT is 6.3 times larger than CANCEL, which is handled with stratified splitting, balanced class weights and macro-averaged metrics.

Limitations of the dataset. The text is synthetic: it was generated from seed texts and expanded with natural language generation, so messages within a category are very consistent. The data is English only, covers general e-commerce support, and has no labels for priority or sentiment. The SUBSCRIPTION category refers to newsletter subscriptions only.

2.1.2 Text Preprocessing using NLTK

NLTK (Natural Language Toolkit) [2], [11] is used to clean and normalise every message before feature extraction. The preprocessing function preprocess_text() applies the steps in Table 2.3. Each step reduces noise or merges different forms of the same word, while words that carry meaning for routing are kept.

Table 2.3: NLTK preprocessing steps

Step	Operation	Purpose
1	Placeholders such as {{Order Number}} → "order number"	Turn dataset placeholders into the words a real customer would type
2	Digits → the word "number"	"#12588" and "#99" carry the same meaning for routing
3	Lowercasing; "can't" → "can not", "won't" → "will not"	"Refund" and "refund" become the same word; negation is kept
4	Tokenization with nltk.word_tokenize	Split the message into words and punctuation
5	"n't" → "not"	Keep negations such as "hasn't arrived"
6	Remove punctuation and symbols (keep words and codes like err-404)	Remove noise without losing technical terms
7	Remove NLTK English stopwords, except not, no, never, nor, cannot, without	Drop filler words but keep words that change the meaning
8	Lemmatization with WordNetLemmatizer (verb, then noun)	"cancelling" → "cancel", "orders" → "order"

Empty or missing input returns an empty string instead of raising an error. Table 2.4 shows real output of the preprocessing function.

Table 2.4: Examples of NLTK preprocessing

Raw customer message	After NLTK preprocessing
I was charged twice for my monthly subscription!!	charge twice monthly subscription
I can't access my account after resetting my password	not access account reset password
hw can I modify purchase {{Order Number}}?	hw modify purchase order number
how can I check bill #12588?	check bill number
I never received a refund for the cancelled orders	never receive refund cancel order
My parcel still hasn't arrived and it's been two weeks	parcel still not arrive two week

2.1.3 Feature Extraction using TF-IDF

A machine learning model needs numbers, not text. The preprocessed message is converted into a vector using **TF-IDF (Term Frequency – Inverse Document Frequency)** [6], [10] with Scikit-

learn's TfidfVectorizer [3]. Each word or two-word phrase (term t) in a message d receives a weight that is high when the term is frequent in the message but rare in the whole training set:

$$tf'(t, d) = 1 + \ln(tf(t, d)) \qquad \text{(sublinear term frequency)}$$

$$idf(t) = \ln((1 + N) / (1 + df(t))) + 1$$

$$w(t, d) = tf'(t, d) \times idf(t), \quad \text{then each vector is scaled to unit length (L2 norm)}$$

Here tf(t, d) is the number of times t occurs in d, N is the number of training messages and df(t) is the number of training messages that contain t. Common words therefore receive low weights and distinctive words such as "refund" or "invoice" receive high weights. The settings used are given in Table 2.5; the fitted vocabulary contains **3,785 terms**.

Table 2.5: TF-IDF settings

Parameter	Value	Reason
ngram_range	(1, 2)	Single words and two-word phrases such as "money back" and "reset password"
min_df	2	Ignore terms that occur in only one training message (mostly typos)
max_features	20,000	Upper limit on the vocabulary size
sublinear_tf	True	Use 1 + ln(tf) so that repeated words do not dominate
preprocessor	preprocess_text (NLTK)	Apply the NLTK pipeline of Section 2.1.2

TF-IDF was chosen because it is simple, fast, needs no GPU, works very well for short keyword-rich messages, and is interpretable. Table 2.6 lists, for each category, the terms with the largest weights in the trained Linear SVM. They match the meaning of each category; a few dataset typos (for example "adress") also appear because the model learned them from the training data.

Table 2.6: Highest-weighted terms per category learned by the Linear SVM

Category	Top terms
ACCOUNT	account, sign-up, signup, user, registration
CANCEL	fee, penalty, charge, withdrawal, termination
CONTACT	talk, speak, contact, chat, agent
DELIVERY	arrive, shipment, period, ship, delivery
FEEDBACK	feedback, claim, leave, file, lodge
INVOICE	number, invoice, bill, person name, name

Category	Top terms
ORDER	order, purchase, acquire, earn, buy
PAYMENT	payment, modality, online, pay, accept
REFUND	refund, reimbursement, rebate, compensation, restitution
SHIPPING	address, delivery, deliveryaddress, set, adress
SUBSCRIPTION	newsletter, subscription, corporate, unsubscribe, subscribe

2.1.4 Classification using Linear SVM

The classification model is a **Linear Support Vector Machine** [4]. Each message is a point in a 3,785-dimensional space (one dimension per TF-IDF term). For every category k the SVM learns a linear decision function, and the category with the highest score is predicted:

$$f_k(x) = w_k \cdot x + b_k ; \qquad \text{predicted category} = \operatorname{argmax}_k \, f_k(x)$$

The weights w_k are chosen so that the boundary (hyperplane) separates the messages of category k from all other messages with the **widest possible margin**, which makes the model robust on new messages. Scikit-learn's LinearSVC [3], which uses the LIBLINEAR solver [12], minimises the regularised squared hinge loss for each category (one-vs-rest, with labels y_i = +1 or −1):

$$\text{minimise} \quad \tfrac{1}{2} \|w_k\|^2 + C \cdot \Sigma_i \, \max(0, 1 - y_i (w_k \cdot x_i + b_k))^2$$

Linear SVMs are well suited to text because text data is high-dimensional and sparse, and documents of different categories are usually linearly separable by their keywords [5]. Two additions are used:

- **Balanced class weights:** errors on small categories (CANCEL, SUBSCRIPTION) are weighted more, in proportion to n / (K · n_c), so that the imbalance does not bias the model towards ACCOUNT.
- **Probability calibration:** because an SVM outputs scores rather than probabilities, the scikit-learn class CalibratedClassifierCV [3] fits a sigmoid (Platt) mapping [7] from scores to probabilities using 3-fold cross-validation on the training data; the final probability is the average over the three calibrated SVMs. The highest probability is shown as the confidence of the prediction and is used for manual-review flagging.

Table 2.7: Linear SVM settings

Setting	Value	Meaning
Model	LinearSVC	Linear Support Vector Machine (one-vs-rest for 11 classes)
C	1.0	Trade-off between a wide margin and fitting every training example
Loss	Squared hinge (default)	Penalty for points inside or on the wrong side of the margin
class_weight	balanced	Gives small categories more weight
Calibration	CalibratedClassifierCV, sigmoid, cv = 3	Converts SVM scores into confidence values

2.2 Validation and Evaluation Design

The model is validated in three stages so that the reported performance reflects how it behaves on messages it has never seen: cross-validation on the training set, scoring on a separate validation set, and a single final evaluation on a held-out test set.

2.2.1 Data Splitting and Cross-Validation

After removing repeated messages, the 24,274 unique messages are split with **stratified sampling** (each split keeps the same category proportions) and a fixed random seed (42), so that the results are reproducible (Table 2.8).

Table 2.8: Data split

Set	Messages	Share	Used for
Training set	16,991	70%	Fitting the pipeline and 5-fold cross-validation
Validation set	3,641	15%	Checking performance on unseen data during development
Test set	3,642	15%	One final, unbiased evaluation
Total	24,274	100%	

In **5-fold stratified cross-validation** [14], the training set is divided into five equal parts; the pipeline is trained on four parts and tested on the fifth, five times, so that every training message

is used for testing exactly once. The mean and standard deviation of the five scores show how stable the model is. Data leakage is prevented by the following design decisions:

- The TF-IDF vectorizer is part of the pipeline, so it is fitted on training data only (also inside every cross-validation fold) and only reused to transform validation and test data.
- Repeated messages are removed before splitting.
- The response and intent columns, which reveal the answer, are never used as input.
- The test set is used only once, after all design decisions were made.

2.2.2 Evaluation Metrics

For each category c, TP, FP and FN denote the true positives, false positives and false negatives of the predictions on the evaluation set. The following metrics are computed from the actual predictions:

$$\text{Accuracy} = \frac{\text{number of correctly classified messages}}{\text{total number of messages}}$$

$$\text{Precision}_c = \frac{TP}{TP + FP} \qquad \text{Recall}_c = \frac{TP}{TP + FN}$$

$$F1_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

$$\text{Macro-F1} = \frac{1}{K} \sum_c F1_c \qquad \text{Weighted-F1} = \sum_c \frac{n_c}{N} \times F1_c$$

- **Accuracy** is the share of all messages classified correctly.
- **Precision** of a category: of the messages the model assigned to it, how many really belong to it (low precision means a team receives tickets that are not theirs).
- **Recall** of a category: of the messages that belong to it, how many the model found (low recall means tickets are lost to other teams).
- **F1-score** is the harmonic mean of precision and recall; it is high only if both are high.
- **Macro average** treats all 11 categories equally, so small categories matter as much as large ones. Because the classes are imbalanced, macro-F1 is used as the main metric. **Weighted average** weights each category by its size n_c.
- The **confusion matrix** shows, for every true category (row), how many messages were predicted as each category (column); off-diagonal cells are errors.

Manual-review rule. In live use, a prediction is flagged for manual review when its confidence is below a threshold (0.50 by default, adjustable in the web application), when none of the words of the message are in the TF-IDF vocabulary, or when the only recognised words are stopwords. Because the model always chooses one of its 11 categories, these rules reduce, but do not remove, the risk of wrong routing.

3. SYSTEM REQUIREMENTS

The system uses classical machine learning and runs entirely on a CPU. It does not need a GPU, a database or any paid or external API.

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Minimum requirement	System used for development
Processor	64-bit dual-core x86-64 CPU	12th Gen Intel Core i5-12450H
Memory (RAM)	4 GB	16 GB
Storage	1 GB free space	About 655 MB used (Python environment 585 MB, NLTK data 50 MB, project files with dataset and model 20 MB)
GPU	Not required	Not used (all training and prediction run on the CPU)
Network	Internet connection for the one-time installation of packages, dataset and NLTK data	Broadband connection
Display	Any screen that can run a modern web browser	Laptop display

3.2 Software Requirements (Libraries used)

Table 3.2: Software and libraries

Software / Library	Version	Purpose
Operating system	Windows 11 (also works on Linux / macOS)	Development and execution platform
Python	3.12.10 (3.10 or newer required)	Programming language
NLTK	3.10.4	Tokenization, stopword list, WordNet lemmatization
NLTK data	punkt_tab, stopwords, wordnet, omw-1.4	Tokenizer models, stopword list and WordNet dictionary
scikit-learn	1.9.1	TF-IDF, Linear SVM, calibration, cross-validation, metrics
pandas	3.1.0	Loading and exploring the dataset
NumPy	2.5.4	Numerical arrays
Matplotlib	3.11.2	Charts

Software / Library	Version	Purpose
Seaborn	0.13.2	Confusion matrix heatmaps
Streamlit	1.66.1	Interactive web application
Joblib	1.6.0	Saving and loading the trained pipeline
Web browser	Any modern browser (Edge, Chrome, Firefox)	Using the web application

The libraries are listed in requirements.txt and installed into a Python virtual environment with the following commands (Windows PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m src.preprocess --setup        # download the NLTK data once
```

4. IMPLEMENTATION

The project is implemented in Python as a set of modules, one per stage of the pipeline. The folder structure is shown below and the responsibility of each module is summarised in Table 4.1.

```
customer-support-classifier/
├── app.py                  Streamlit web application
├── requirements.txt        Python dependencies
├── dataset/                Bitext CSV file
├── models/                 ticket_classifier.joblib (saved pipeline), test_split.csv
├── reports/                results.json, classification report, confusion matrix
└── src/
    ├── config.py           paths, random seed, split sizes, TF-IDF settings
    ├── data_loader.py      Module 1: loading, validation, exploration, cleaning
    ├── preprocess.py       Module 2: NLTK preprocessing
    ├── train.py            Modules 3-4: TF-IDF + Linear SVM pipeline, validation
    ├── evaluate.py         Module 5: metrics, reports, confusion matrix
    └── predict.py          Module 6: live prediction and manual-review flags
```

Table 4.1: Project modules

Module	File	Responsibility
1. Data loading and exploration	src/data_loader.py	Download and load the CSV, check columns, report missing values, duplicates and class distribution, clean the data
2. NLP preprocessing	src/preprocess.py	NLTK tokenization, stopword removal (keeping negations) and lemmatization; NLTK data setup
3. Feature extraction	src/train.py	TF-IDF vectorizer with the NLTK preprocessing plugged in
4. Model training	src/train.py	Linear SVM with calibration; stratified split, cross-validation, validation, saving
5. Evaluation	src/evaluate.py	Accuracy, precision, recall, F1, classification report, confusion matrix, error list
6. Live classification	src/predict.py	Input validation, prediction, confidence and manual-review flags
7. Web application	app.py	Streamlit interface with five pages

4.1 Pseudocode / Algorithm

Algorithm 1 – Training and evaluation (src/train.py)

```
Input : Bitext CSV file
Output: saved pipeline P, evaluation results
1.  D <- read CSV;  check that columns 'instruction' and 'category' exist
2.  D <- keep (instruction, category); drop rows with empty text or label
3.  D <- drop repeated messages (case- and whitespace-insensitive)
4.  (Train, Val, Test) <- stratified split of D into 70% / 15% / 15%, seed = 42
5.  P <- Pipeline( TF-IDF(preprocessor = PreprocessText, ngrams = (1,2), min_df = 2),
                   Calibrated( LinearSVC(C = 1, class_weight = balanced), cv = 3 ) )
6.  for i = 1 to 5 do                                    // 5-fold cross-validation
7.      fit a copy of P on 4 folds of Train; score accuracy and macro-F1 on fold i
8.  report mean and standard deviation of the 5 fold scores
9.  fit P on Train;  compute metrics on Train and on Val
10. predict Test with P (used only once)
11. compute accuracy, precision, recall, F1 (macro, weighted), per-class report,
    confusion matrix and list of misclassified messages
12. save P to ticket_classifier.joblib and all results to reports/
```

Algorithm 2 – NLTK preprocessing (PreprocessText, src/preprocess.py)

```
Input : raw message m          Output: cleaned text
1.  if m is missing or empty then return ""
2.  replace {{Placeholder Name}} by 'placeholder name';  replace digit runs by 'number'
3.  m <- lowercase(m);  expand "can't" -> "can not", "won't" -> "will not"
4.  T <- nltk.word_tokenize(m)
5.  for each token t in T do
6.      if t = "n't" then t <- "not"
7.      skip t if it is punctuation / symbol
8.      skip t if t is an NLTK stopword and not in {not, no, never, nor, cannot, without}
9.      t <- WordNetLemmatizer.lemmatize(t, verb);  t <- lemmatize(t, noun)
10.     append t to output
11. return output tokens joined by spaces
```

Algorithm 3 – Classifying a new ticket (src/predict.py)

```
Input : message m, saved pipeline P, threshold θ (default 0.50)
Output: category, confidence, review flag
1.  if m is empty or longer than 2000 characters then report an input error
2.  K <- terms of PreprocessText(m) that exist in the TF-IDF vocabulary of P
3.  p <- P.predict_proba(m)            // NLTK -> TF-IDF -> Linear SVM -> calibration
4.  category <- argmax(p);  confidence <- max(p)
5.  review <- (K is empty) or (K contains only stopwords) or (confidence < θ)
6.  return category, confidence, review
```

4.2 Output (Screenshots)

The complete pipeline is executed with the command python -m src.train. Figure 4.1 shows its actual console output: dataset cleaning, the data split, the 5-fold cross-validation scores, the validation scores and the final test-set results. (The script also prints an additional duplicate check and a check on 20 hand-written messages, which are not discussed further in this report.)

```
Model: NLTK preprocessing -> TF-IDF -> Linear SVM
[1/7] Dataset: 26872 rows -> 24274 after cleaning (2598 repeated messages removed)
[2/7] Split: train=16991  validation=3641  test=3642  (11 categories)
[3/7] 5-fold cross-validation on the training set...
      macro-F1 per fold: [0.9981, 0.9989, 0.998, 0.9989, 0.9985]
      mean macro-F1: 0.9985 (+/- 0.0004)
[4/7] Training on the training set and scoring on the validation set...
      train accuracy: 0.9998   validation accuracy: 0.9975   validation macro-F1: 0.9973
[5/7] Evaluating on the held-out test set...
      accuracy          0.9978
      macro_precision   0.9986
      macro_recall      0.9971
      macro_f1          0.9978
      weighted_f1       0.9978
      weakest categories: [{'category': 'SUBSCRIPTION', 'f1': 0.9932}, {'category': 'PAYMENT', 'f1': 0.995},
{'category': 'ACCOUNT', 'f1': 0.9969}]
[6/7] Duplicate-leakage check...
      ... (output of the additional duplicate check and hand-written check omitted) ...
[7/7] Saved model to D:\aaNLP_AAT\customer-support-classifier\models\ticket_classifier.joblib
      Saved evaluation results to D:\aaNLP_AAT\customer-support-classifier\reports
```

Figure 4.1: Console output of the training and evaluation script (python -m src.train)

Results of validation and testing. Table 4.2 lists the cross-validation scores of every fold and Table 4.3 compares the training, validation and test results.

Table 4.2: 5-fold cross-validation on the training set

Fold	1	2	3	4	5	Mean ± std
Accuracy	0.9979	0.9988	0.9982	0.9985	0.9985	0.9984 ± 0.0003
Macro-F1	0.9981	0.9989	0.9980	0.9989	0.9985	0.9985 ± 0.0004

Table 4.3: Training, validation and test results

Data	Accuracy	Macro precision	Macro recall	Macro-F1	Weighted-F1
Training set (16,991)	0.9998	0.9999	0.9998	0.9999	0.9998
Validation set (3,641)	0.9975	0.9975	0.9971	0.9973	0.9975
Test set (3,642)	0.9978	0.9986	0.9971	0.9978	0.9978

The five fold scores are almost identical (standard deviation 0.0004), so the model is stable. The validation and test scores are very close to the training score, so the model is not overfitting. On the held-out test set the model classified 3,634 of 3,642 messages correctly (**accuracy 99.78%, macro-F1 0.9978**). Table 4.4 gives the per-category results.

Table 4.4: Per-category results on the test set

Category	Precision	Recall	F1-score	Support
ACCOUNT	0.9950	0.9988	0.9969	804
CANCEL	1.0000	1.0000	1.0000	141
CONTACT	1.0000	0.9966	0.9983	295
DELIVERY	1.0000	0.9960	0.9980	247
FEEDBACK	1.0000	0.9966	0.9983	298
INVOICE	1.0000	1.0000	1.0000	270
ORDER	0.9957	1.0000	0.9978	460
PAYMENT	0.9966	0.9933	0.9950	298
REFUND	0.9974	1.0000	0.9987	387
SHIPPING	1.0000	1.0000	1.0000	293
SUBSCRIPTION	1.0000	0.9866	0.9932	149
Macro average	0.9986	0.9971	0.9978	3,642
Weighted average	0.9978	0.9978	0.9978	3,642

The weakest categories are SUBSCRIPTION (F1 = 0.9932) and PAYMENT (F1 = 0.9950). Figure 4.2 shows the confusion matrix: almost all messages lie on the diagonal. Table 4.5 lists all 8 misclassified test messages. Every error contains a misspelled keyword or two words joined without a space; the key word is then not in the vocabulary, and the model has to rely on the remaining words.

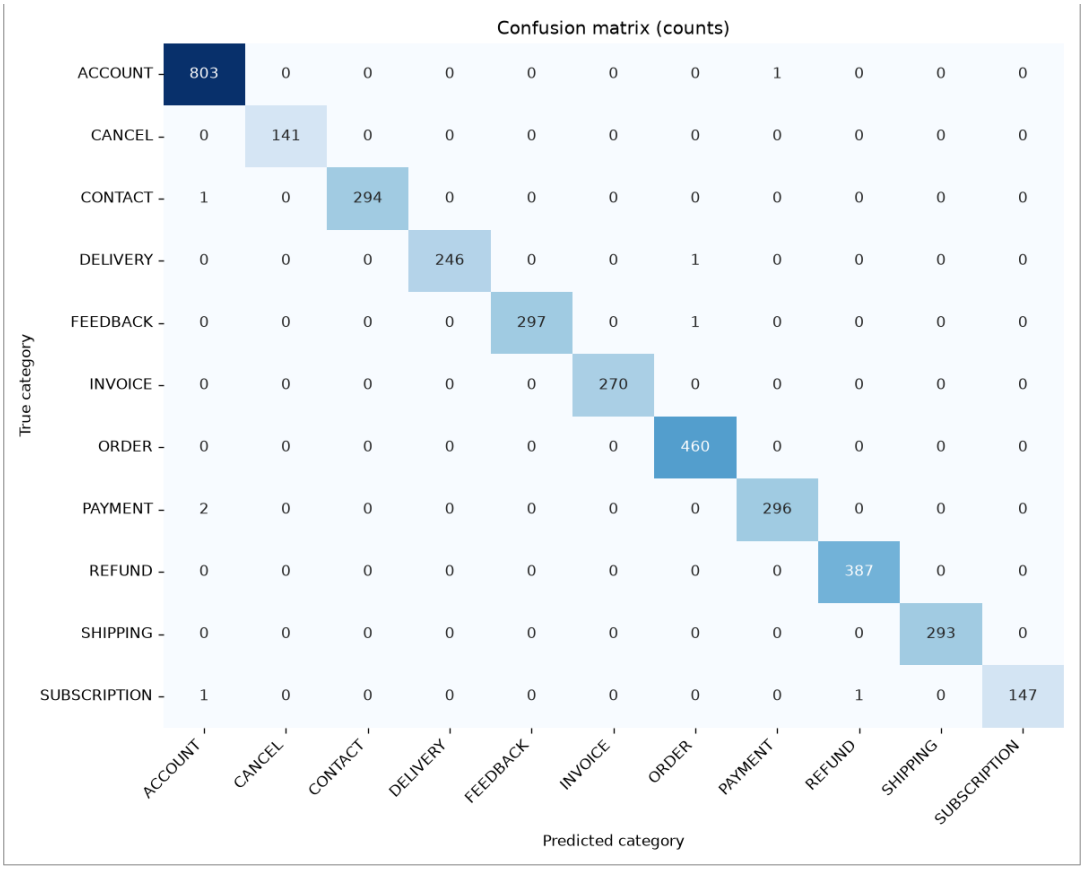


Figure 4.2: Confusion matrix of the Linear SVM on the test set (rows: true category, columns: predicted category)

Table 4.5: All misclassified test messages

Message	True	Predicted
where do I leavefeedback about your products?	FEEDBACK	ORDER
I want assistance to see when will my purchase arrtive	DELIVERY	ORDER
help to receive your newslettyer	SUBSCRIPTION	REFUND
assistance informing of an error with onlinepayment	PAYMENT	ACCOUNT
need help reporting errors with registratons	ACCOUNT	PAYMENT
i want assistance canceling the newslettersubscription	SUBSCRIPTION	ACCOUNT
contactingassistant	CONTACT	ACCOUNT
help me inform of a problem withpayment	PAYMENT	ACCOUNT

Web application. The Streamlit application (started with streamlit run app.py) loads the saved pipeline and the saved evaluation results; it never retrains the model. The following screenshots were taken from the running application.

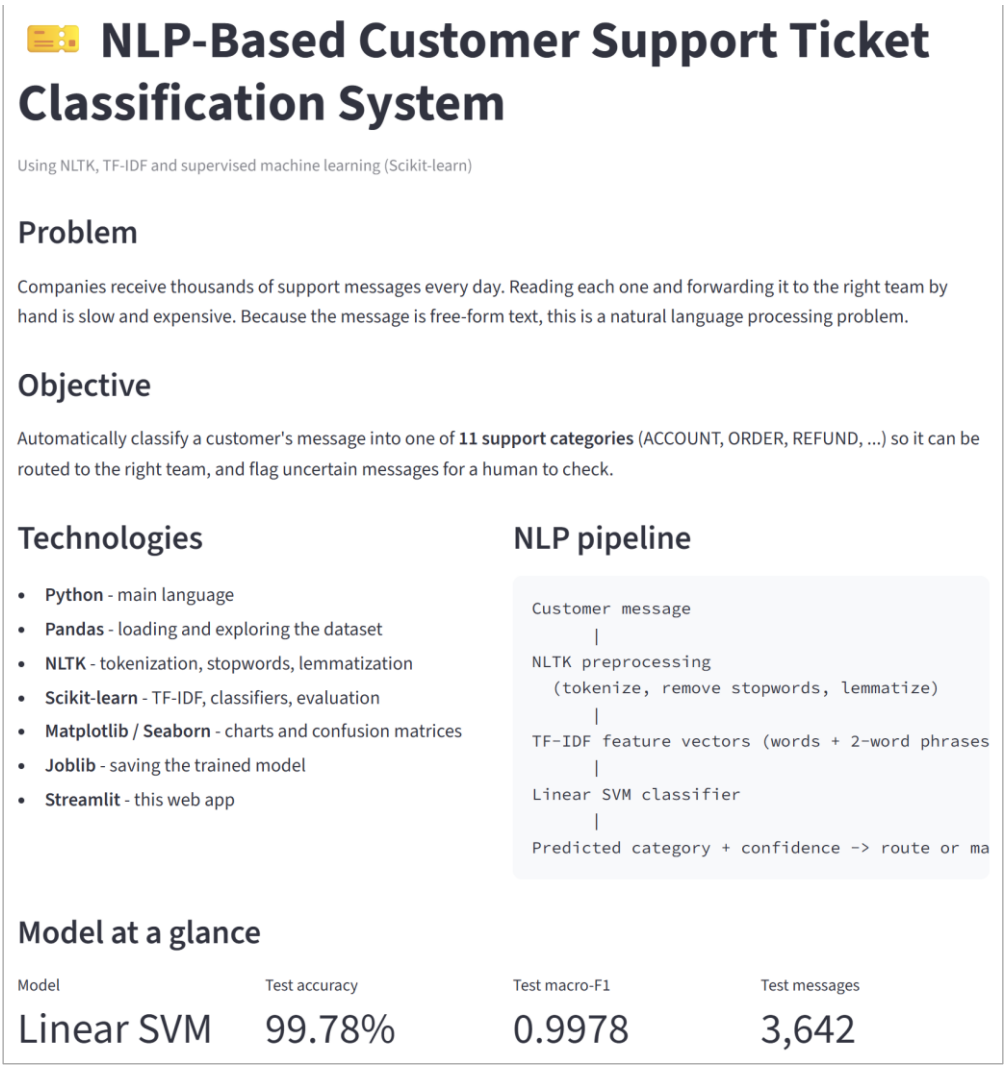
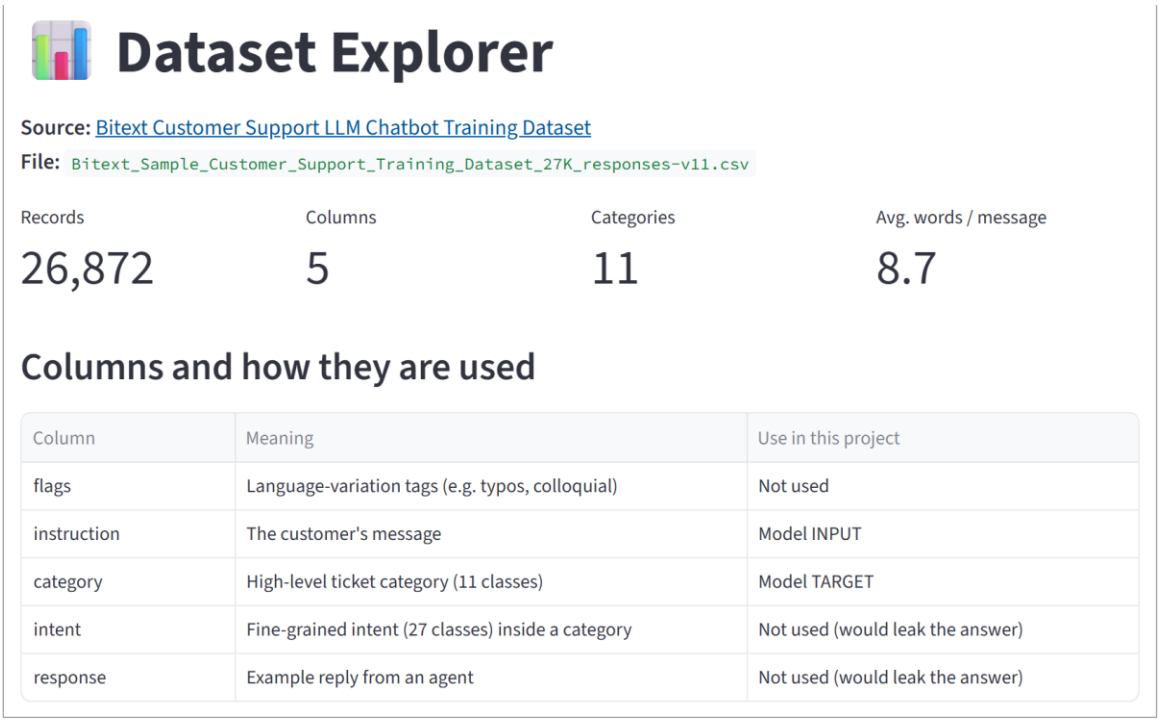


Figure 4.3: Home page: problem, objective, technologies and NLP pipeline



Dataset Explorer

Source: Bitext Customer Support LLM Chatbot Training Dataset

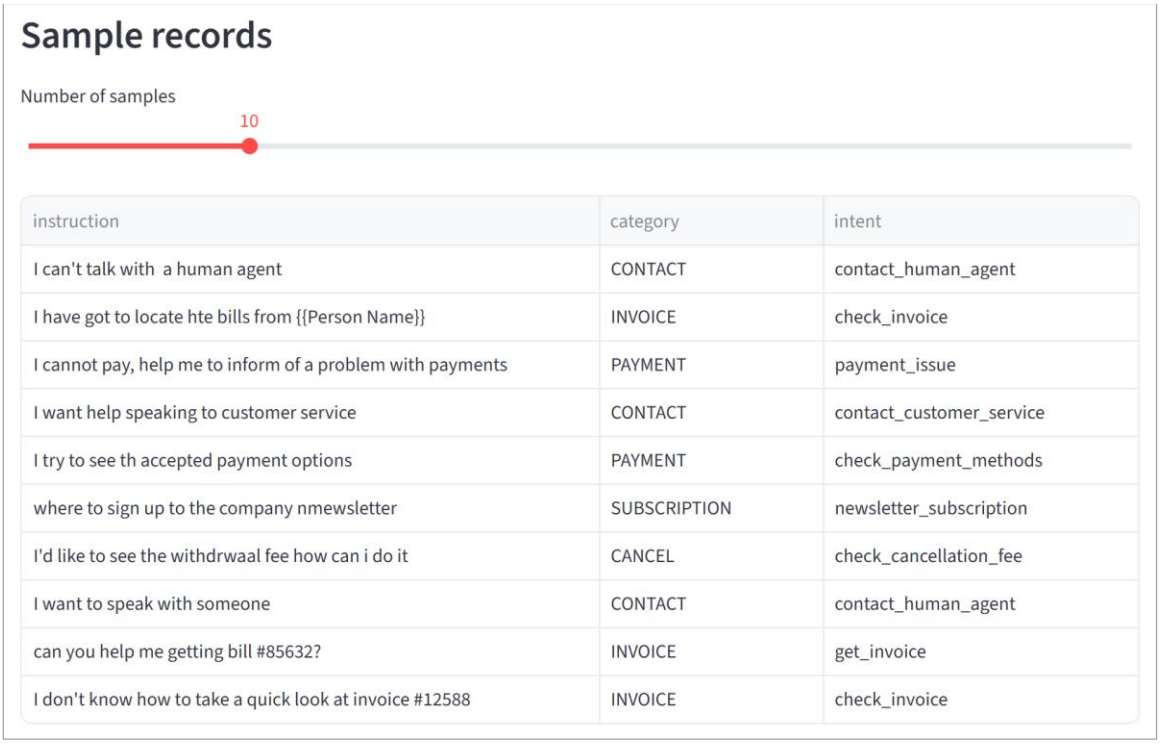
File: `Bitext_Sample_Customer_Support_Training_Dataset_27K_responses-v11.csv`

Records	Columns	Categories	Avg. words / message
26,872	5	11	8.7

Columns and how they are used

Column	Meaning	Use in this project
flags	Language-variation tags (e.g. typos, colloquial)	Not used
instruction	The customer's message	Model INPUT
category	High-level ticket category (11 classes)	Model TARGET
intent	Fine-grained intent (27 classes) inside a category	Not used (would leak the answer)
response	Example reply from an agent	Not used (would leak the answer)

Figure 4.4: Dataset Explorer: dataset size and use of each column



Sample records

Number of samples

10

instruction	category	intent
I can't talk with a human agent	CONTACT	contact_human_agent
I have got to locate hte bills from {{Person Name}}	INVOICE	check_invoice
I cannot pay, help me to inform of a problem with payments	PAYMENT	payment_issue
I want help speaking to customer service	CONTACT	contact_customer_service
I try to see th accepted payment options	PAYMENT	check_payment_methods
where to sign up to the company nmewsletter	SUBSCRIPTION	newsletter_subscription
I'd like to see the withdrwaal fee how can i do it	CANCEL	check_cancellation_fee
I want to speak with someone	CONTACT	contact_human_agent
can you help me getting bill #85632?	INVOICE	get_invoice
I don't know how to take a quick look at invoice #12588	INVOICE	check_invoice

Figure 4.5: Dataset Explorer: sample records with category and intent

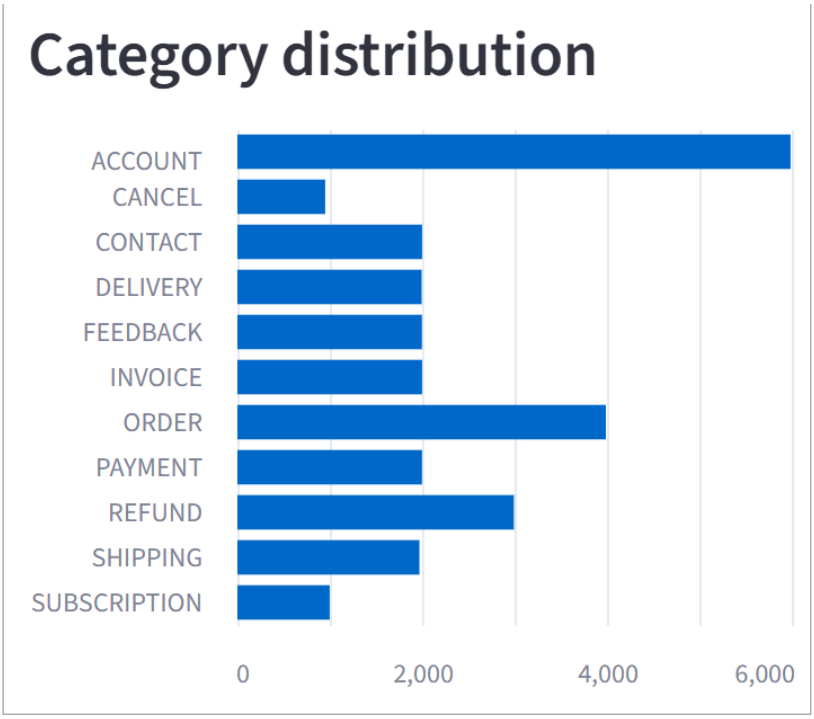


Figure 4.6: Dataset Explorer: category distribution chart

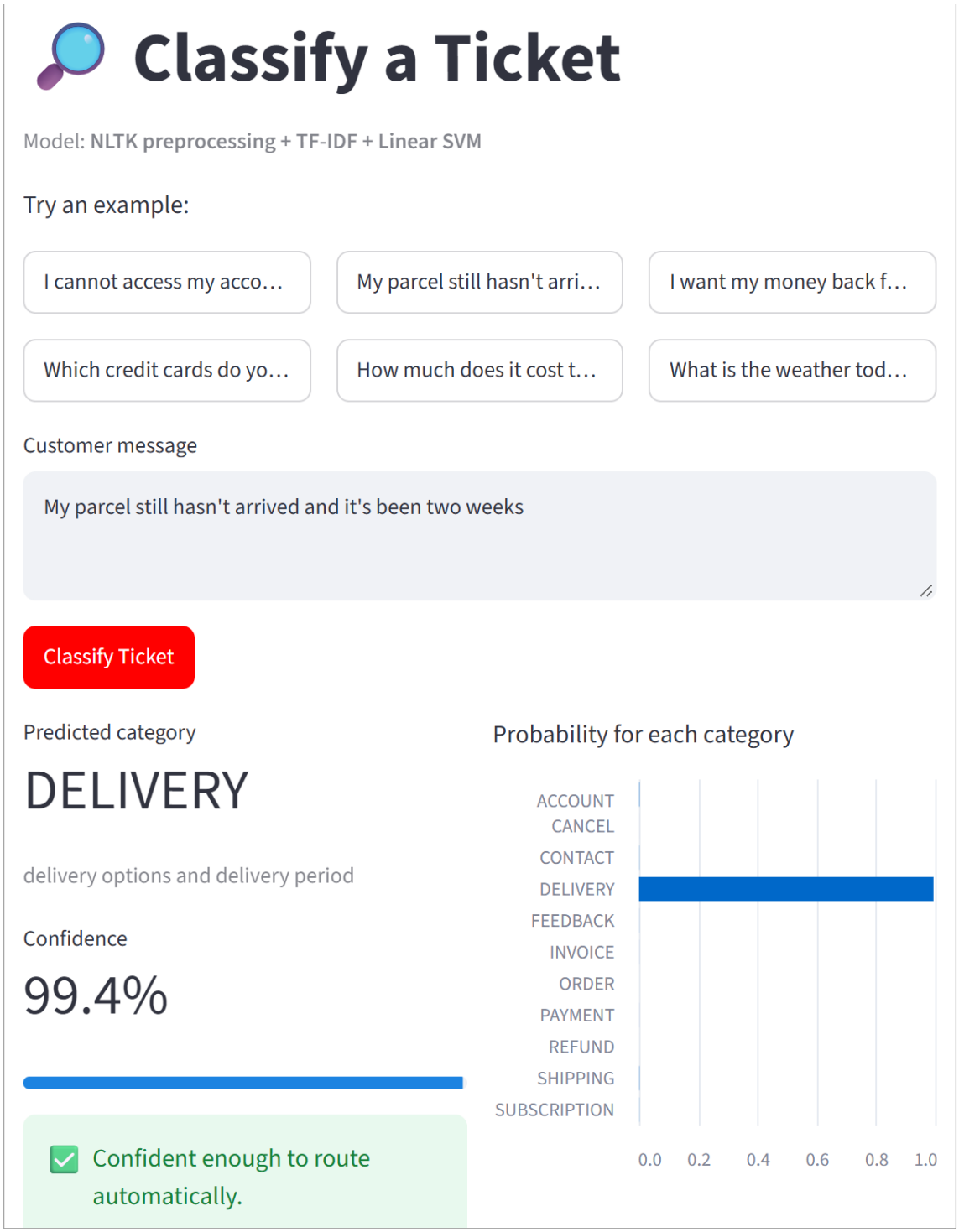


Figure 4.7: Classify a Ticket: a new message is classified as DELIVERY with 99.4% confidence

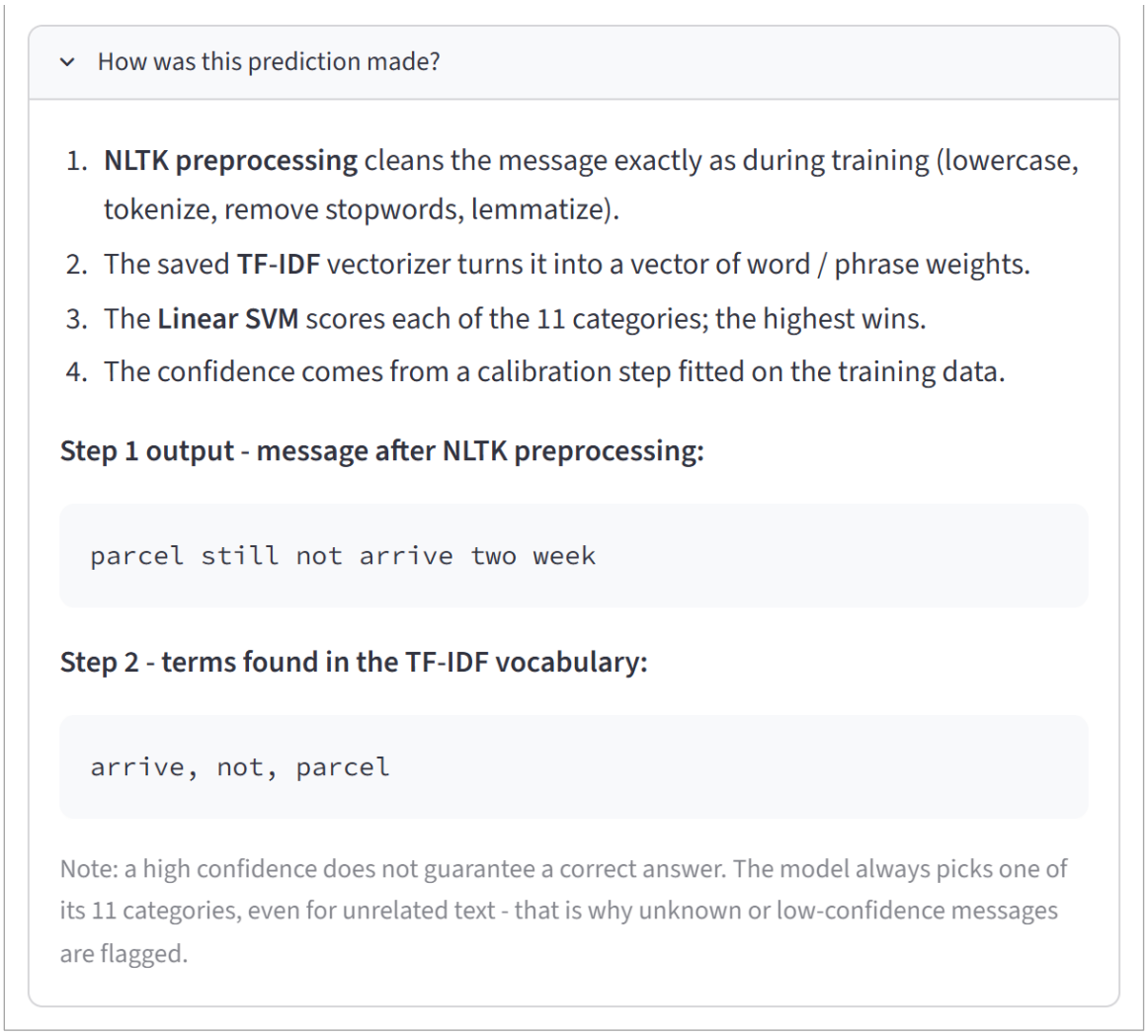


Figure 4.8: Prediction explanation: output of NLTK preprocessing and the recognised TF-IDF terms

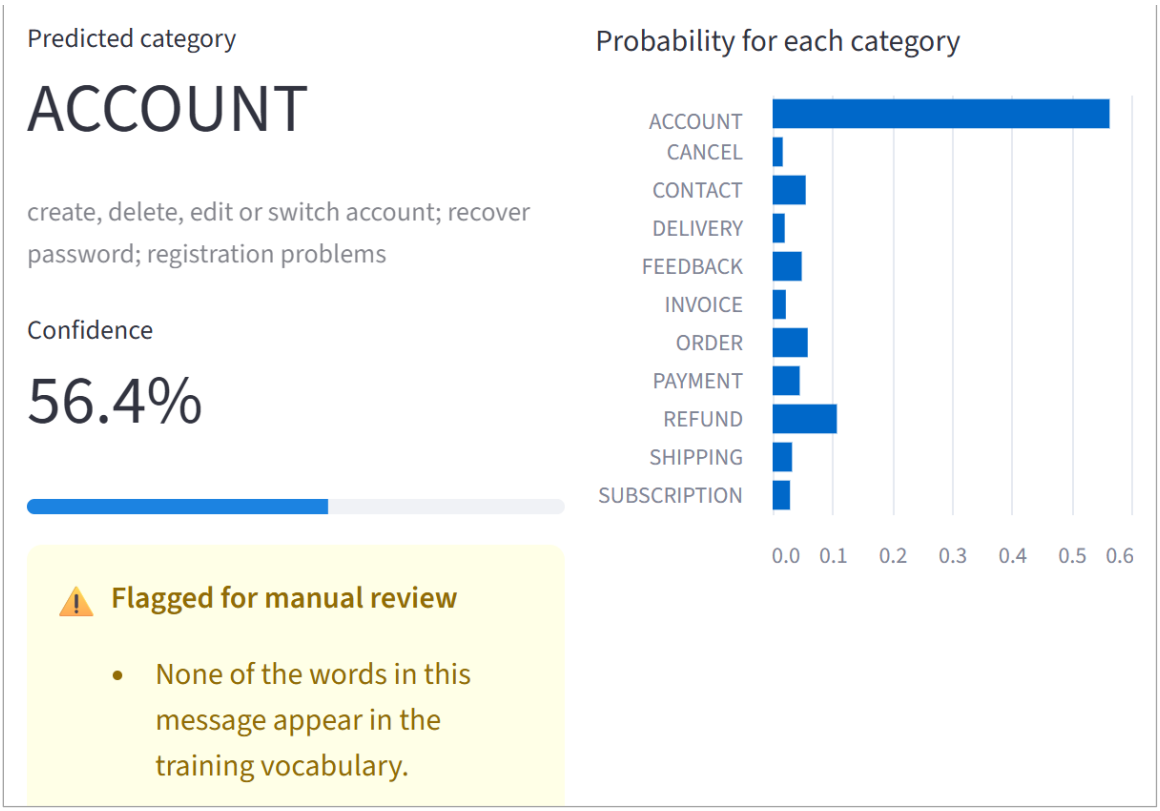


Figure 4.9: An unrelated message ("What is the weather today?") is flagged for manual review

Table 4.6 summarises the live predictions made in the application for new messages written by us (these messages are not part of the dataset).

Table 4.6: Live predictions for new messages

Message	Predicted	Confidence	Status
I cannot access my account after resetting my password.	ACCOUNT	99.7%	Auto-routed
My parcel still hasn't arrived and it's been two weeks	DELIVERY	99.4%	Auto-routed
I want my money back for the broken headphones	REFUND	99.2%	Auto-routed
Which credit cards do you accept?	PAYMENT	98.4%	Auto-routed
How much does it cost to terminate my contract early?	CANCEL	98.8%	Auto-routed
What is the weather today?	ACCOUNT	56.4%	Manual review (no known words)
(empty message)	–	–	Rejected with an error message

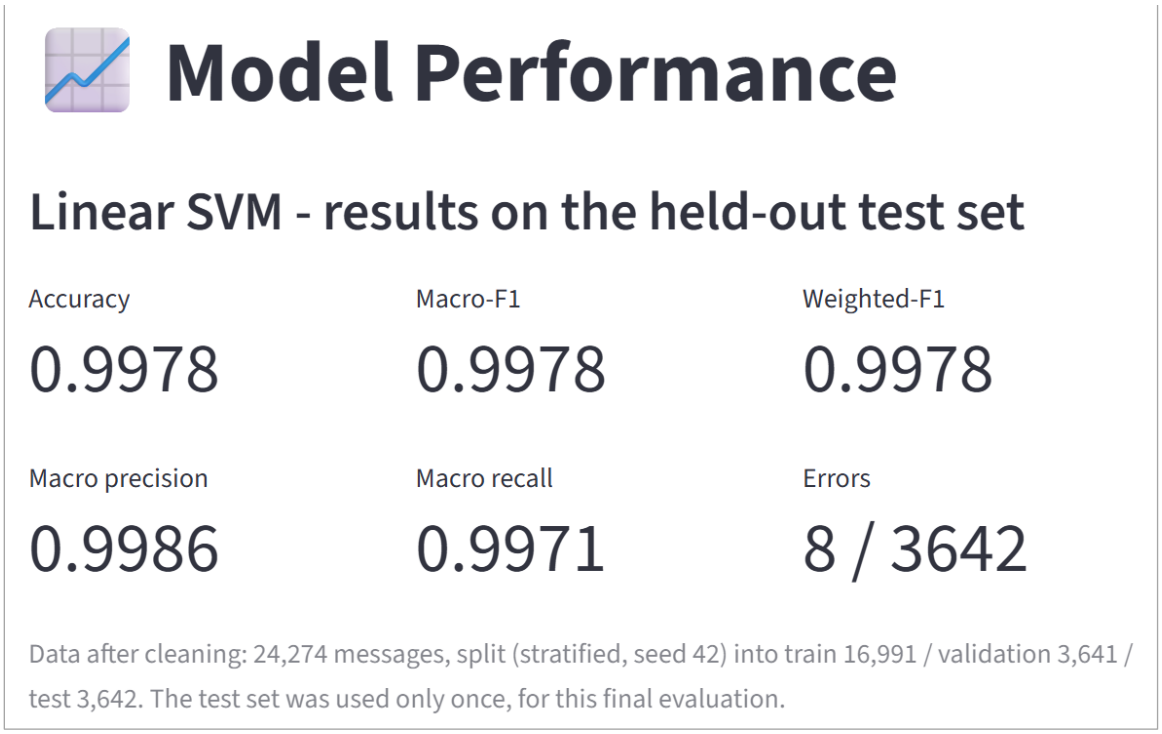


Figure 4.10: Model Performance: test-set results of the Linear SVM

Validation | Per-category report | Confusion matrix | Error analysis

5-fold cross-validation on the training set: the training data is split into 5 parts; the model is trained on 4 and tested on the remaining one, 5 times.

fold	accuracy	macro-F1
1	0.9979	0.9981
2	0.9988	0.9989
3	0.9982	0.998
4	0.9985	0.9989
5	0.9985	0.9985

Mean accuracy **0.9984** (± 0.0003), mean macro-F1 **0.9985** (± 0.0004).

Training vs validation vs test (a small gap means the model is not overfitting):

data	accuracy	macro precision	macro recall	macro-F1	weighted-F1
Training set	0.9998	0.9999	0.9998	0.9999	0.9998
Validation set	0.9975	0.9975	0.9971	0.9973	0.9975
Test set	0.9978	0.9986	0.9971	0.9978	0.9978

Figure 4.11: Model Performance: cross-validation and training / validation / test comparison

Validation | Per-category report | Confusion matrix | Error analysis

category	precision	recall	f1-score	support
ACCOUNT	0.995	0.9988	0.9969	804
CANCEL	1	1	1	141
CONTACT	1	0.9966	0.9983	295
DELIVERY	1	0.996	0.998	247
FEEDBACK	1	0.9966	0.9983	298
INVOICE	1	1	1	270
ORDER	0.9957	1	0.9978	460
PAYMENT	0.9966	0.9933	0.995	298
REFUND	0.9974	1	0.9987	387
SHIPPING	1	1	1	293
SUBSCRIPTION	1	0.9866	0.9932	149
macro avg	0.9986	0.9971	0.9978	3642
weighted avg	0.9978	0.9978	0.9978	3642

Weakest categories: SUBSCRIPTION (0.9932), PAYMENT (0.9950), ACCOUNT (0.9969)

Figure 4.12: Model Performance: per-category classification report

Validation | Per-category report | Confusion matrix | Error analysis

Every misclassified message in the test set:

message	true	predicted
where do I leavefeedback about your products?	FEEDBACK	ORDER
I want assistance to see when will my purchase arrtive	DELIVERY	ORDER
help to receive your newslettyer	SUBSCRIPTION	REFUND
assistance informing of an error with onlinepayment	PAYMENT	ACCOUNT
need help reporting errors with registratons	ACCOUNT	PAYMENT
i want assistance canceling the newslettersubscription	SUBSCRIPTION	ACCOUNT
contactingassistant	CONTACT	ACCOUNT
help me inform of a problem withpayment	PAYMENT	ACCOUNT

Most frequent confusions:

true	predicted	count
PAYMENT	ACCOUNT	2
FEEDBACK	ORDER	1
DELIVERY	ORDER	1
SUBSCRIPTION	REFUND	1
ACCOUNT	PAYMENT	1

Figure 4.13: Model Performance: error analysis of misclassified test messages

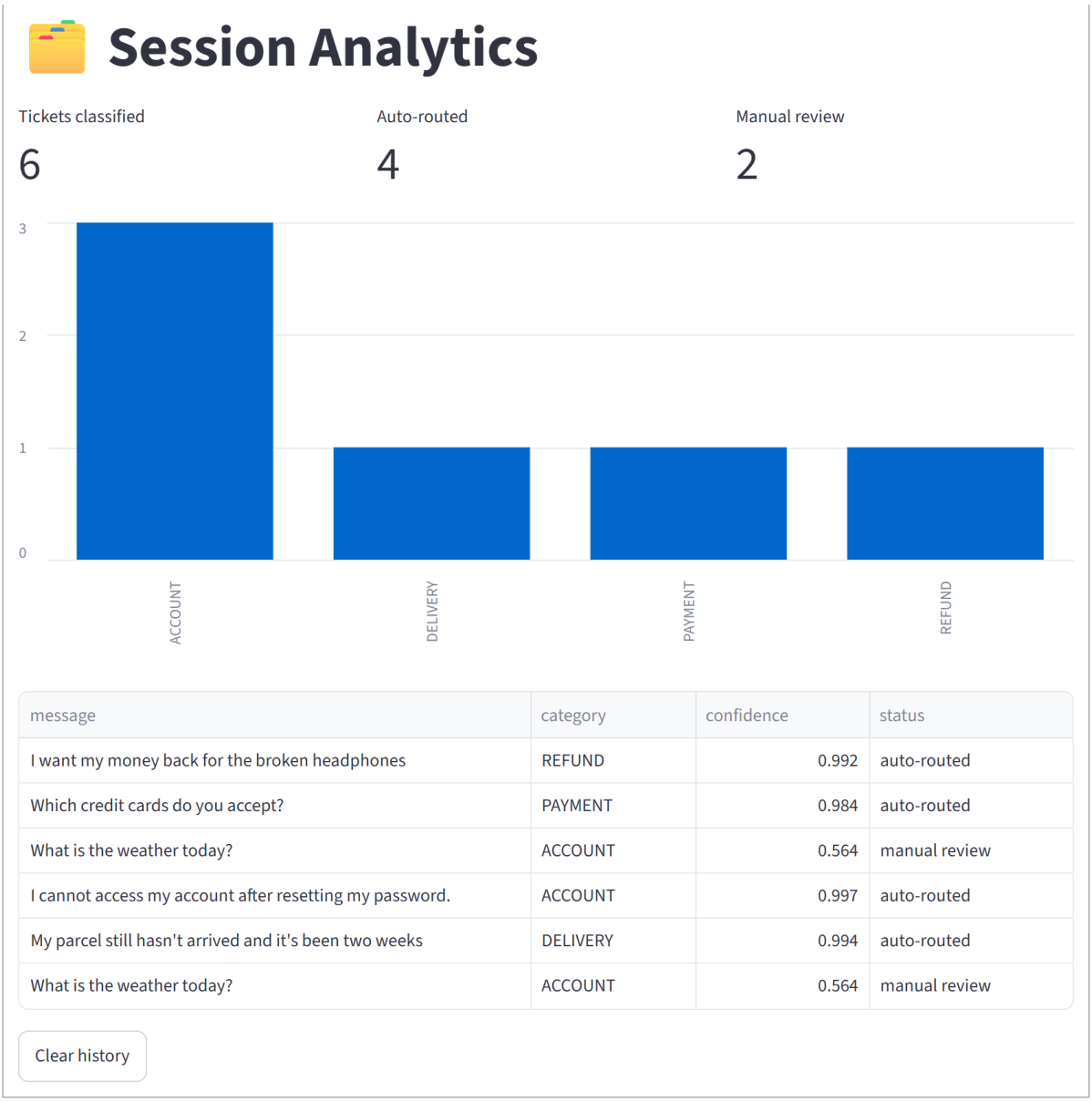


Figure 4.14: Session Analytics: summary of the tickets classified during a session

5. CONCLUSION

This work developed an NLP-based customer support ticket classification system that reads a customer's message and predicts which of 11 support categories should handle it. The system combines an NLTK preprocessing pipeline (tokenization, stopword removal that keeps negations, and WordNet lemmatization), TF-IDF feature extraction with words and two-word phrases, and a Linear Support Vector Machine with calibrated confidence scores. All three stages are combined into one saved pipeline, so new messages are processed exactly like the training data.

The model was trained on the original Bitext Customer Support dataset after removing 2,598 repeated messages, and it was validated without data leakage. 5-fold cross-validation gave a mean macro-F1 of 0.9985 ± 0.0004, and the final evaluation on 3,642 unseen test messages gave an **accuracy of 99.78% and a macro-F1 of 0.9978**, with only 8 errors, all caused by misspelled or joined keywords. The Streamlit web application demonstrates the complete system: dataset exploration, live classification with a confidence score and an explanation of the NLP steps, manual-review flagging of uncertain messages, and the model's validation results.

Limitations. The dataset is synthetic and template-based, so real customer tickets, which are longer and noisier, are likely to give lower accuracy. TF-IDF considers word order only within two-word phrases, misspelled or joined keywords become unknown words, each ticket receives only one category, and the category definitions follow the dataset (for example, SUBSCRIPTION means newsletter subscriptions only).

Future scope. The system can be improved by:

- evaluating and fine-tuning the model on real, labelled tickets from an organisation;
- adding spelling correction or character n-gram features to handle typos and joined words;
- using word embeddings or a fine-tuned transformer model to better understand paraphrases;
- supporting several labels per ticket and predicting the 27 fine-grained intents as a second stage;
- using the decisions made during manual review to retrain the model regularly.

REFERENCES

[1] Bitext, "Bitext – Customer Service Tagged Training Dataset for LLM-based Virtual Assistants," GitHub repository. [Online]. Available: https://github.com/bitext/customer-support-llm-chatbot-training-dataset

[2] S. Bird, E. Klein and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O'Reilly Media, 2009.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[5] T. Joachims, "Text categorization with Support Vector Machines: Learning with many relevant features," in Proc. 10th European Conference on Machine Learning (ECML), LNCS vol. 1398, Springer, 1998, pp. 137–142.

[6] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[7] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[8] G. A. Miller, "WordNet: A lexical database for English," Communications of the ACM, vol. 38, no. 11, pp. 39–41, 1995.

[9] D. Jurafsky and J. H. Martin, Speech and Language Processing, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.

[10] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, UK: Cambridge University Press, 2008.

[11] NLTK Project, "NLTK Documentation." [Online]. Available: https://www.nltk.org

[12] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang and C.-J. Lin, "LIBLINEAR: A library for large linear classification," Journal of Machine Learning Research, vol. 9, pp. 1871–1874, 2008.

[13] Streamlit Inc., "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io

[14] Scikit-learn developers, "scikit-learn User Guide." [Online]. Available: https://scikit-learn.org/stable/user_guide.html